Project Audit: NHS Personal Demographics Service (PDS)

Test Plan - Version 1.0

1. Executive Summary

Objective:

To validate the correctness, reliability, and security of key **NHS PDS FHIR API** endpoints—specifically `Patient Search` and `Patient Read`—by verifying response accuracy, data integrity, error handling, and authentication behavior under both valid and invalid conditions. The goal is to ensure compliance with **HL7 FHIR R4** standards and safe integration with third-party healthcare applications.

Business Value:

Accurate patient identification is critical in healthcare systems. This audit reduces the risk of clinical errors caused by incorrect or ambiguous patient data, ensures the protection of sensitive medical and psychological information, and supports efficient and reliable clinician workflows.

2. Scope of Testing

This audit focuses on the **core backend functionality** of the **NHS PDS FHIR API**, specifically the validation of patient identification workflows through API interactions.

In Scope

- **Patient Search (`Patient Search Endpoint`):**
  Validation of search behavior using `family name`, `gender`, and `birthdate`, including:
  - Exact match vs. partial match behavior
  - Case sensitivity (e.g., `"smith"` vs `"Smith"`)
  - Handling of multiple result sets

    - No-result scenarios and appropriate API responses
- **Patient Data Retrieval (`Patient Read Endpoint`):**
  Validation of full patient record retrieval using a valid `NHS Number`, including:
    - Response structure and field completeness
    - Data consistency across responses
    - Conformance to **FHIR JSON schema**
- **Error Handling & Response Codes:**
  Verification of correct HTTP status codes and error responses, including:
    - `400 Bad Request` (invalid input)
    - `401 Unauthorized` (authentication failures)
    - `404 Not Found` (non-existent resources)
    - `422 Unprocessable Entity` (semantic validation errors)
- **Authentication & Security Behavior:**
  Validation of API access control mechanisms:
    - Missing or invalid API keys
    - Expired or malformed tokens
    - Unauthorized access attempts
- **Internationalization (i18n):**
  Validation of system support for UTF-8 characters and international naming conventions (e.g., `"Díaz"`, `"O'Neill"`)

Out of Scope

- **UI / Frontend Testing:**
  This project focuses exclusively on backend API validation.
- **Performance & Load Testing:**
  No stress or load testing will be conducted. Only basic response-time observations in the sandbox environment are included.

- **Database-Level Validation:**
  Direct database verification is not included due to restricted access to backend systems.

3. Technical Stack & Environment

Environment

- **Primary Environment:** NHS Digital API Sandbox (Mock Data)
- **Data Type:** Synthetic patient data (non-PII) for safe testing
- **Limitations:**
  - No access to production environment
  - Limited control over backend data states
  - Performance metrics not representative of real-world load

Standards & Protocols

- **Healthcare Standard:** HL7® FHIR® R4
- **Data Format:** JSON (FHIR-compliant resource structure)
- **Authentication:** API Key / OAuth 2.0 (depending on endpoint configuration)

Tools & Usage

- **Postman**
  Used for:
  - API request execution (GET, query parameters, headers)
  - Writing automated test scripts using JavaScript (`pm.test`, `pm.response`)
  - Environment and collection variable management
  - Response validation (status codes, schema checks, data assertions)
  - Debugging using Postman Console (`console.log`)
- **Jira**
  Used for:

    - Defect tracking and lifecycle management
    - Prioritization (Severity vs Priority classification)
    - Documentation of reproducible bugs with evidence
- **Notion**
  Used for:
    - Test case documentation
    - Test execution tracking
    - Final QA portfolio presentation

Test Execution Approach

- Tests are executed manually via Postman collections
- Assertions are automated using JavaScript within Postman test scripts
- Test data is parameterized using environment variables to improve reusability
- Results are validated through:
    - Response body inspection
    - Status code verification
    - Console debugging logs

4. Test Strategy (The "Senior" Approach)

Testing Type	Strategy	Goal
Functional	Positive testing using valid NHS numbers and query parameters. Validate full response payload.	Ensure correct patient data is returned and matches expected FHIR structure and values.
Negative	Boundary Value Analysis and invalid inputs (e.g., malformed NHS numbers, invalid dates, special characters).	Ensure the API rejects invalid input with correct status codes and error messages.

Testing Type	Strategy	Goal
Security	Token validation (missing, invalid, expired). Validate authorization scopes and access control.	Ensure unauthorized requests are blocked (`401`) and forbidden actions return (`403`).
Response Validation (API Core)	Validate JSON response structure against FHIR schema. Verify required fields (`id` , `name` , `birthDate`) and data types.	Ensure API contract integrity and data consistency.
Error Handling & Message Validation	Validate error responses for clarity, consistency, and correctness.	Ensure error messages are actionable, standardized, and non-ambiguous for developers and clinicians.
UX/Psychology (API-Level)	Analyze readability and usability of error messages and system feedback.	Ensure responses reduce confusion and support fast clinical decision-making.

5. Risk Assessment (Enhanced with Technical Depth + Psychology Insight)

Risk 1: Multiple Patient Matches (Ambiguous Identity)

- **Impact:** High
  Incorrect patient selection may lead to severe clinical errors (e.g., wrong medication or treatment).
- **Technical Cause:**
  Broad search parameters returning large datasets without ranking or prioritization.
- **Mitigation (Testable):**
  - Validate response includes **multiple distinct patient IDs**
  - Verify **result count and pagination behavior**
  - Assert presence of distinguishing attributes (e.g., `birthDate` , `gender`)

- Ensure API does not truncate or omit critical identifying fields

Risk 2: Slow Response Time (> 2 seconds)

- **Impact:** High
  Delays in emergency scenarios can affect clinical decision-making and increase stress.
- **Technical Cause:**
  Backend latency, inefficient queries, or network delays.
- **Mitigation (Testable):**
  - Add Postman assertions:

    ```
    pm.expect(pm.response.responseTime).to.be.below(2000);
    ```

  - Monitor response time trends across multiple requests

Risk 3: Incorrect or Incomplete Patient Data (Data Integrity)

- **Impact:** Critical
  Missing or incorrect data (e.g., wrong birthdate) can lead to misdiagnosis or incorrect treatment.
- **Technical Cause:**
  Data mapping errors, backend inconsistencies, or partial responses.
- **Mitigation (Testable):**
  - Validate presence of mandatory fields (`id`, `name`, `birthDate`)
  - Verify data types and formats (FHIR compliance)
  - Cross-check values against expected mock dataset

Risk 4: Unauthorized Data Access (Security Breach)

- **Impact:** Critical
  Exposure of sensitive patient data violates privacy laws and ethical standards.
- **Technical Cause:**
  Weak authentication or authorization controls.

- **Mitigation (Testable):**
  - Send requests without token → expect `401`
  - Send invalid token → expect `401`
  - Send valid token with insufficient scope → expect `403`

Risk 5: Unexpected API Response Structure (Contract Break)

- **Impact:** High
  Client applications may fail if response structure changes unexpectedly.
- **Technical Cause:**
  Backend updates without version control or schema validation.
- **Mitigation (Testable):**
  - Validate JSON schema structure
  - Assert presence and type of key fields
  - Detect missing or additional unexpected fields

6. Acceptance Criteria (Definition of Done)

The API is considered **production-ready** when the following conditions are met:

- ✅ **Successful Responses:**
  All valid API requests return appropriate **2xx status codes** (`200`, `201`, or `204`) with accurate and complete data.
- ✅ **Error Handling:**
  Invalid inputs (e.g., numeric values in name fields, malformed dates) consistently return appropriate **4xx status codes** (`400`, `401`, `404`, `422`) with clear and descriptive error messages.
- ✅ **Schema Compliance:**
  All API responses conform to the **FHIR R4 JSON schema**, including:
  - Presence of required fields (`id`, `name`, `birthDate`)
  - Correct data types and formats
- ✅ **Test Coverage:**
  All **in-scope endpoints and scenarios** (functional, negative, security) are

covered by executed test cases.

- ✅ **Pass Rate Threshold:**
  At least **95% of test cases pass**, with **0 critical defects open**.
- ✅ **Defect Management:**
  All identified defects are:
  - Logged in **Jira**
  - Include clear **steps to reproduce**
  - Properly classified by **severity and priority**
- ✅ **Performance Baseline:**
  API response time is consistently **below 2 seconds** under normal test conditions (sandbox environment).

7. Requirements Traceability Matrix (RTM)

To ensure 100% test coverage and alignment between Business Requirements and Technical Execution.

Requirement ID	Requirement Description	Linked Test Case IDs	Status	Jira / Defect
REQ-PDS-SRCH-01	Basic Patient Search (Name & DOB)	TC-PDS-01, 02, 04	✅ Passed	-
REQ-PDS-SRCH-02	Refined Identity Search (Gender/Filters)	TC-PDS-05	⚠️ Blocked	Sandbox Limit
REQ-PDS-SEC-01	OAuth2 Token & Auth Validation	TC-PDS-03, 12, 13	✅ Passed	-
REQ-PDS-VAL-02	i18n (Accented Character) Support	TC-PDS-11	✅ Passed	-
REQ-PDS-UPD-01	Patient Record Update (General)	TC-PDS-06, 07	⚠️ Blocked	Sandbox Limit

REQ-PDS-UPD-02	Concurrency Control (Optimistic Locking/ETags)	TC-PDS-08, 09	✅ Passed	-
REQ-PDS-SEC-04	Schema Validation & Input Sanitization	TC-PDS-10	❌ Failed	**BUG-PDS-001**

8. Test Data Strategy

Ensuring controlled, secure, and comprehensive use of test data in a regulated healthcare environment.

8.1 Data Source

- NHS Digital **Synthetic Data Set** (Sandbox environment)
- All data is mock and designed for safe API testing

8.2 Data Privacy & Compliance

- No real **PII (Personally Identifiable Information)** is used
- All testing complies with data protection standards such as:
  - **GDPR**
  - Healthcare data handling best practices (HIPAA-aligned principles)

8.3 Test Data Classification

Data Type	Description	Example
Valid Data	Correct and expected inputs	NHS ID `9000000009`
Invalid Data	Incorrect format or values	NHS ID `ABC123`, special characters in numeric fields
Boundary Data	Edge limits of valid inputs	Very old DOB, future DOB
Edge Cases	Uncommon but valid scenarios	Names: “Díaz”, “O'Neill”

Data Type	Description	Example
Security Data	Authentication variations	Missing / Invalid / Expired tokens

8.4 Key Test Entities

- **Primary Patient:**
  NHS ID `9000000009` (Standard valid patient)
- **Restricted Record:**
  NHS ID `9912003071` (Sensitive access scenario)
- **Edge Case Inputs:**
  - Names with accents and special characters
  - Partial name searches
  - Multiple match scenarios

8.5 Data Usage Strategy

- Reuse stable test patients for consistency across test cases
- Combine static data (known NHS IDs) with dynamic query parameters
- Ensure each test case clearly defines:
  - Input data
  - Expected output based on that data

9. Defect Management Lifecycle

Structured process for identifying, documenting, prioritizing, and resolving defects to ensure software quality and release stability.

9.1 Defect Workflow (Lifecycle Stages)

Each defect follows a standardized lifecycle:

1. **New** → Bug is created and logged
2. **Triaged** → Reviewed and prioritized by QA/Product
3. **Assigned** → Allocated to a developer

4. **In Progress** → Developer is working on the fix
5. **Resolved** → Fix implemented
6. **Retested** → QA validates the fix
7. **Closed** → Bug is verified and completed

👉 Optional states:

- **Rejected** (not a bug / expected behavior)
- **Deferred** (postponed to future release)

9.2 Priority vs Severity (Critical distinction)

Type	Meaning	Example
Severity	Impact on the system	Wrong patient data returned
Priority	Urgency to fix	Must be fixed before release

👉 Example:

- A typo = Low severity but could be High priority (visible to users)
- Data mismatch = High severity + High priority

9.3 Priority Classification

- **P1 (Critical):**
  Data integrity issues (e.g., incorrect patient data returned)
  🚫 *Release blocker*
- **P2 (High):**
  Security failures (e.g., unauthorized access allowed)
- **P3 (Medium):**
  Functional issues (e.g., filters not working correctly)
- **P4 (Low):**
  Minor issues (e.g., unclear error messages)

9.4 Defect Documentation Standard

All defects logged in **Jira** must include:

- **Title:** Clear and concise summary
- **Environment:** Sandbox / API version
- **Steps to Reproduce**
- **Expected Result**
- **Actual Result**
- **Status Code & Response Body** (VERY important for API testing)
- **Severity & Priority**
- **Attachments:**
  - Postman Console logs
  - Request/Response screenshots

9.5 QA Ownership & Quality Control

- QA is responsible for:
  - Accurate defect reporting
  - Clear reproduction steps
  - Validation after fix (retesting)
- No defect is closed without:
  - Verified fix
  - Regression impact check (if applicable)

10. Test Deliverables

Artifacts produced during the testing lifecycle to ensure traceability, transparency, and quality assurance.

10.1 Planning & Design Artifacts

- **Test Plan Document**
  Defines scope, strategy, risks, and approach for the API audit.

- **Test Cases (Notion)**
  Detailed test scenarios including:
  - Preconditions
  - Steps
  - Expected Results
  - Linked Requirement IDs (RTM)
- **Requirements Traceability Matrix (RTM)**
  Ensures full coverage between requirements and test cases.

10.2 Execution Artifacts

- **Postman Collection**
  Contains:
  - API requests
  - Automated test scripts (`pm.test`)
  - Environment variables
  - Response validations
- **Test Execution Report**
  Summary of test results including:
  - Total test cases executed
  - Passed / Failed
  - Blocked tests
  - Key defects identified

10.3 Defect & Quality Artifacts

- **Bug Reports (Jira format)**
  Fully documented defects with:
  - Steps to reproduce
  - Expected vs Actual results

- Severity & Priority
- Supporting evidence

10.4 Final Deliverables (Portfolio-Ready)

- **API Test Report / Case Study (Notion)**
  Final summary including:
  - Testing scope
  - Key findings
  - Risk analysis
  - Recommendations for improvement

NHS Test Suite

Defect Log

Executive Summary